Assignment 2: Loopback Test

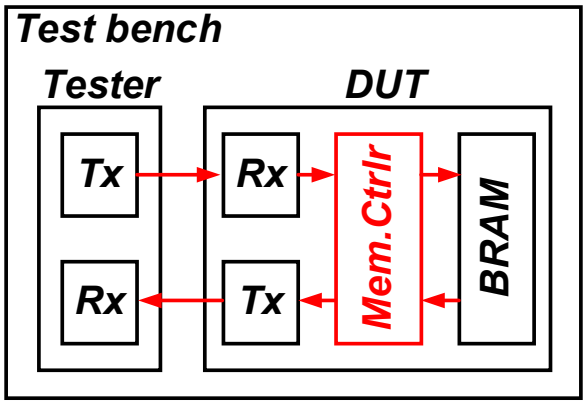
Due Date

- **Board Implementation – April 15(Mon) & April 17(Wed) In-Class**
- **Report – April 20 (Sun), 11:59 PM**

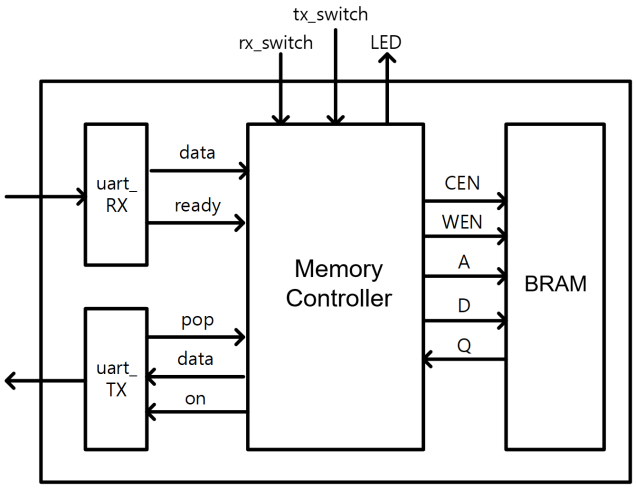
Notice

- Ensure that there are no Verilog HDL syntax errors.
- Score for the 2nd Assignment will be based on the Board Implementation and the Report.
- Design the Loopback Test using the skeleton code provided.
- **DO NOT** modify the I/O ports of the top module without modifying XDC file, as it may cause errors during implementation.
- **Generate the bitstream file in Vivado** before attending the board implementation classes.

In Term Project 2-1, we will design "memory loopback" module. This is the basic component for debugging the circuit implementation. UART and BRAM, which were explained during the lectures, will be utilized for the project. However, you don't need to implement or modify the UART design from the given skeleton (modification is possible but not recommended). BRAM should be implemented strictly following the options on page 3. The one you should design is only *memory_control.v* which stores the restored data from UART_Rx to BRAM or reads the data from BRAM and transmits them through UART_TX by handling the signals of UART and BRAM.



After the reset signal (Active HIGH for this project), *rx_switch* will make the module start receiving. Following the sequence of data from the tester, the data should be stored from address 0 of BRAM until *MEMORY_DEPTH*, a parameter that represents the number of data packets to be transmitted from the tester. After the data has been received, the output port LED turns on. Tester will check the led and then turns off the *rx_switch* and turns on the tx_switch. At that moment, the *memory_loopback* module should re-transmit the data in the same sequence of receiving.

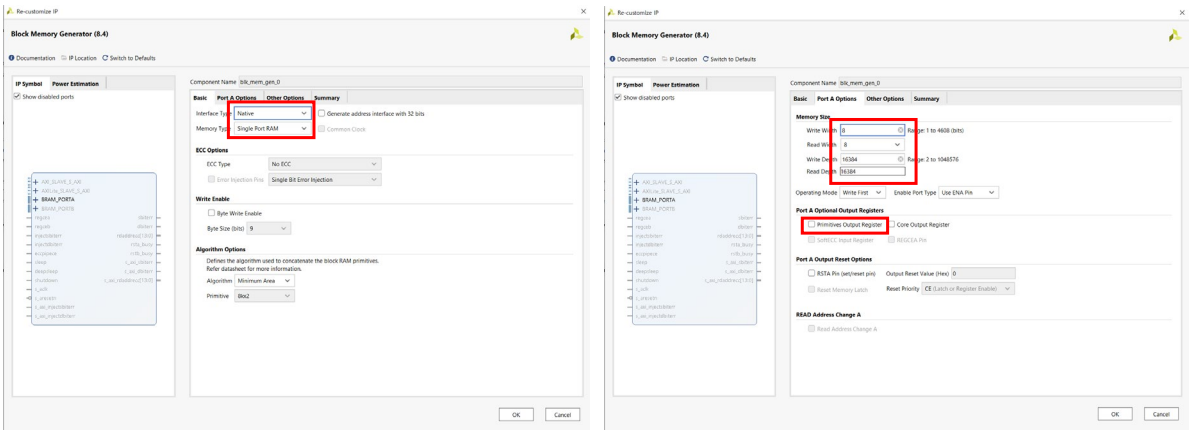


The testbench and tester modules are provided for this project.You can verify your design by adding the 4 files in the sim folder as simulation source in the VIVADO project. Please use the parameter *MEMORY_DEPTH* to define the range of data to be stored and transmitted. The goal of this project is to transmit and receive image data with a resolution of 128x128. However, running the simulation with whole image data will take a lot of time. Thus, your testbench file only tests data transmission of 100 8-bit packets as defined in the given testbench. The board implementation will be conducted using the full-size image, thus, generate the BRAM with Depth of 16384. Please generate the BRAM following the guide on the next page. Any variation in BRAM operation caused by not following the guideline will result in a deduction during grading.

Here are some tips for coding in Verilog:

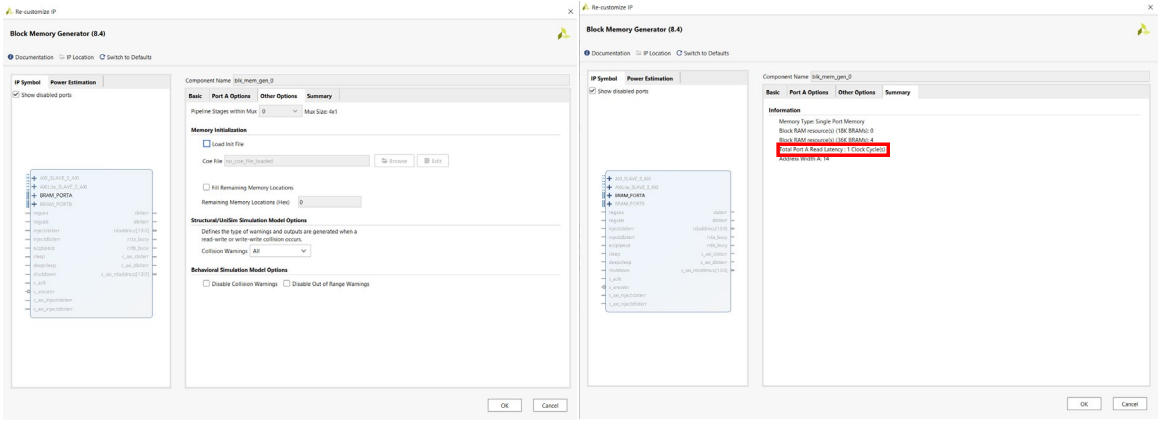
1. **Avoid assigning multiple *reg* variables in the same *always* loop**. The compiler may synthesize latches for undefined cases, which can lead to errors. For beginners, it is highly recommended to use **a separate *always* loop for each *reg* variable.**
2. **Ensure all variable cases are defined.**
   - Every ***if*** and ***else if*** condition should be **followed by an *else*.**
   - Every ***case*** statement should include a ***default*** case.
3. **Do not mix blocking (=) and non-blocking (<=) assignments in the same always loop or for the same variable.**

Block Memory Generator Guide



Check “Native” and “Single Port RAM” for interface and memory type.

Fill out depth as “8” and width as “16384”. Turn off the primitive output register option.

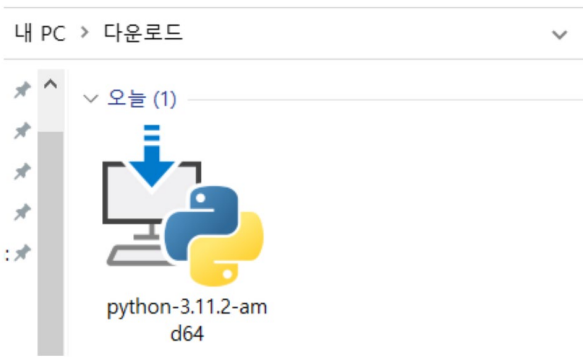


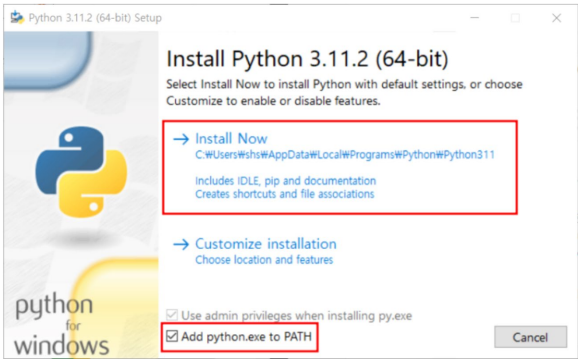
Do not modify anything in *Other Options* page. You can check the memory configuration at the summary. The read delay will be 1 clock cycle. It means the dout will be driven after 1 clock period from input signal. However, please check the operation of BRAM by simulation before designing the memory loopback.

Jupyter Notebook Installation Guide

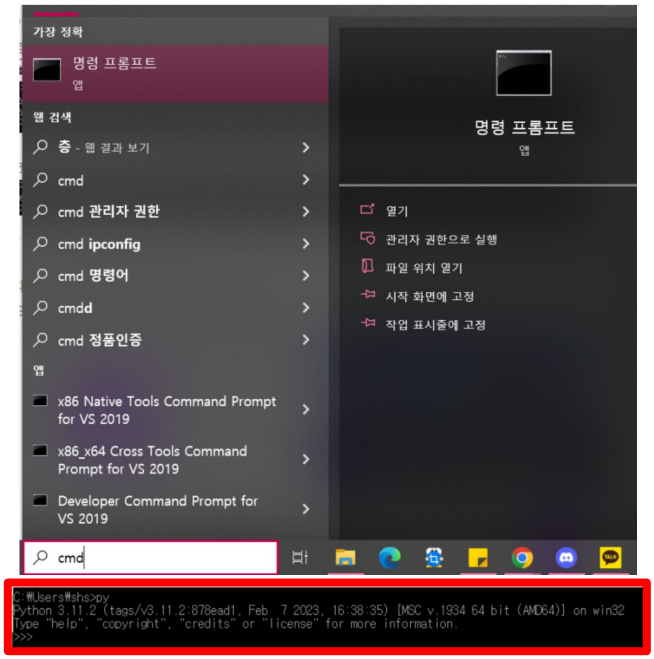
During the board implementation process, the FPGA board communicates with the PC via UART. Therefore, the PC used for verification must have a program set up for UART communication. For simple serial communication testing, we plan to use the Pyserial library, and we recommend installing Jupyter Notebook for this purpose. The installation instructions are provided below.

1. Download python3 installer following below link.
   https://www.python.org/downloads/

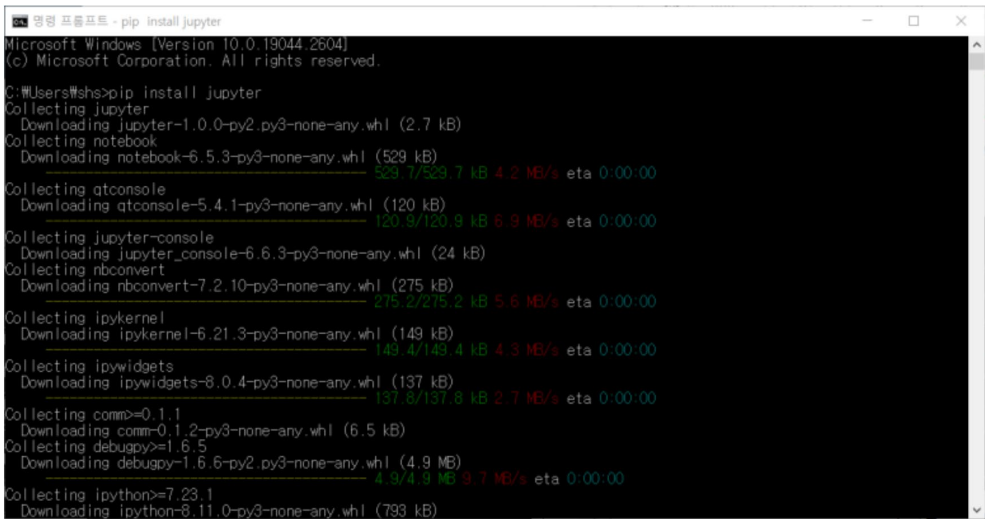




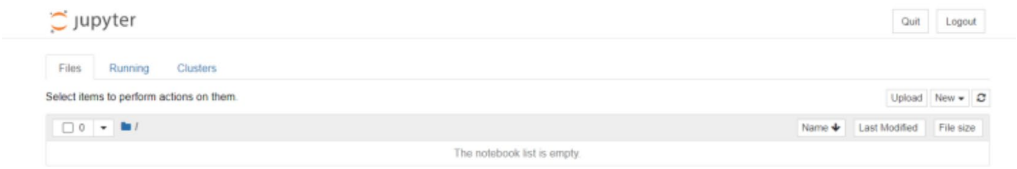
2. Check whether python is successfully installed.



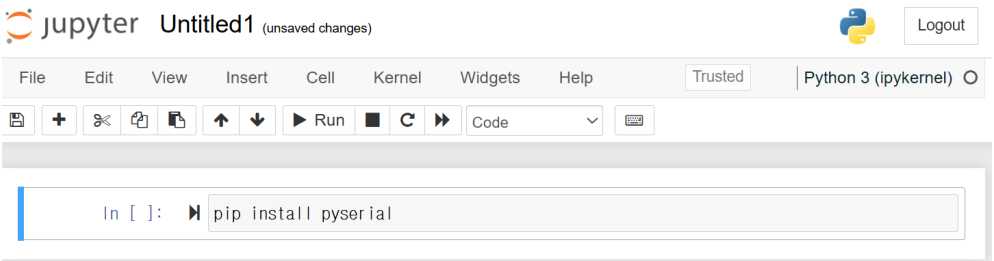
3. Type pip install jupyter in cmd prompt. You will see the message below.



4. Start Jupyter Notebook by typing:
jupyter notebook –notebook-dir='YOUR PROJECT PATH'.
jupyter notebook will be opened on web browser.



5. Create a new kernel and type *pip install pyserial*.



It is highly recommended to complete the following guide for installing Jupyter Notebook before attending the board implementation class.